

Introduction to File Organization, Storage & Indexes in DBMS

DBMS Lecture Series

Unit-5

Dr. Pavinder Yadav

Assistant Professor
School of Computer Science
University of Petroleum and Energy Studies
Dehradun



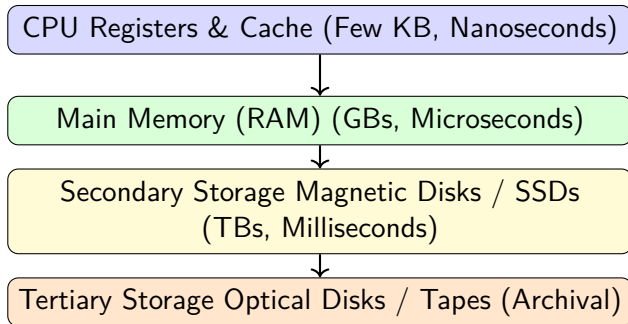
- 1 Introduction to Storage in DBMS
- 2 Secondary Storage Devices
- 3 Buffering of Blocks and Placing File Records on Disk
- 4 Operations on Files
- 5 File Organisation - Heap, Sorted and Hash
- 6 RAID Technology
- 7 Indexing in DBMS
 - Example: Building a 3-Order B-Tree
- 8 Types of Indexes

- 1 Introduction to Storage in DBMS
- 2 Secondary Storage Devices
- 3 Buffering of Blocks and Placing File Records on Disk
- 4 Operations on Files
- 5 File Organisation - Heap, Sorted and Hash
- 6 RAID Technology
- 7 Indexing in DBMS
 - Example: Building a 3-Order B-Tree
- 8 Types of Indexes

- DBMS must store **large volumes of data** efficiently on **secondary storage**.
- Memory hierarchy:
 - **Primary storage:** Main memory (RAM) – fast, volatile.
 - **Secondary storage:** Magnetic disks, SSDs – non-volatile, slower.
 - **Tertiary storage:** Optical disks, tapes – archival.
- DBMS manages data through:
 - 1 **Files** – collection of related records.
 - 2 **Pages/Blocks** – fixed-size units of data transfer between disk and main memory.

Key Idea

A **block** is the smallest unit of data transfer between *disk* and *buffer (RAM)*.



Observation

Cost ↓ and **Capacity** ↑ as we go down the hierarchy, but **Access time** ↑.

- Disk access is **much slower** than main memory.
- Performance of a DBMS heavily depends on:
 - 1 How data is organized on disk.
 - 2 How efficiently it is retrieved into main memory.
- Concepts like:
 - **Buffering of Blocks**
 - **Heap / Sorted / Hashed files**
 - **RAID (Parallel Disk Access)**

help in optimizing I/O cost.

Important

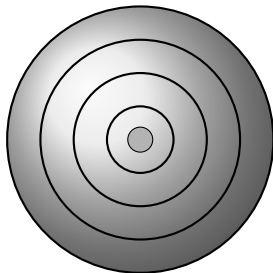
Every SQL query execution ultimately depends on *how data is stored and accessed physically*.

- 1 Introduction to Storage in DBMS
- 2 Secondary Storage Devices**
- 3 Buffering of Blocks and Placing File Records on Disk
- 4 Operations on Files
- 5 File Organisation - Heap, Sorted and Hash
- 6 RAID Technology
- 7 Indexing in DBMS
 - Example: Building a 3-Order B-Tree
- 8 Types of Indexes

- Secondary storage provides **non-volatile** and **large-capacity** storage for databases.
- Common secondary storage devices:
 - 1 Magnetic Disks (HDDs)
 - 2 Solid-State Drives (SSDs)
 - 3 Optical Disks (CD, DVD)
 - 4 Magnetic Tapes (for backup)
- DBMS primarily relies on **Magnetic Disks** for storage due to their cost-effectiveness and random access capability.

Key Idea

A **disk** consists of several platters coated with magnetic material, each with two recording surfaces.



Head Movement
(Seek)



Disk Platters with
Concentric Tracks

- **Track:** A circular path on the disk surface.
- **Sector:** A subdivision of a track; smallest addressable unit on disk.
- **Cylinder:** Set of tracks at the same radius across all platters.

- Accessing a block from disk involves:
 - 1 **Seek Time (T_s)**: Time to move the disk arm to the track containing the block.
 - 2 **Rotational Latency (T_r)**: Time waiting for the desired sector to rotate under the read/write head.
 - 3 **Transfer Time (T_t)**: Time to actually transfer data.
- **Total Access Time:**

$$T_{access} = T_s + T_r + T_t$$

- Typical values:
 - Seek time: 2–10 ms
 - Rotational latency: 4 ms (average)
 - Transfer time: 0.5 ms

Performance Note

Disk I/O is millions of times slower than main memory access — hence the importance of buffering and caching.

- SSDs use **flash memory** instead of magnetic platters.
- No mechanical parts — hence:
 - Very low access latency.
 - Higher throughput.
 - Better shock resistance.
- However, SSDs are:
 - More expensive per GB.
 - Have limited write cycles.

	Feature	HDD	SSD
Comparison Summary	Access Time	5–15 ms	0.1–0.5 ms
	Sequential Speed	Moderate	High
	Durability	Mechanical	No moving parts
	Cost/GB	Low	High

- 1 Introduction to Storage in DBMS
- 2 Secondary Storage Devices
- 3 Buffering of Blocks and Placing File Records on Disk**
- 4 Operations on Files
- 5 File Organisation - Heap, Sorted and Hash
- 6 RAID Technology
- 7 Indexing in DBMS
 - Example: Building a 3-Order B-Tree
- 8 Types of Indexes

- When a DBMS retrieves data from disk, it is stored temporarily in a **buffer** (main memory).
- The **Buffer Manager** is responsible for:
 - ① Transferring disk blocks to and from main memory.
 - ② Managing which blocks are currently in memory.
 - ③ Replacing old blocks when buffer space is full.
- Typical replacement strategies:
 - **LRU (Least Recently Used)** – removes block not used recently.
 - **MRU (Most Recently Used)** – opposite of LRU.
 - **FIFO (First-In First-Out)** – oldest block removed first.

Definition

A **Buffer Block** is a copy of a disk block currently residing in main memory for processing.

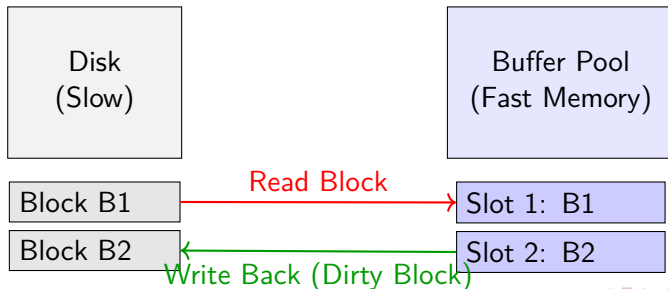
Example

When executing:

```
SELECT * FROM Employee WHERE EmpID = 101;
```

- 1 DBMS checks if the required block is already in buffer.
- 2 If not, it reads the block from disk into buffer.
- 3 The record is processed directly from buffer (faster).

- ➊ **Request Block:** Query requests a particular block.
- ➋ **Check Buffer:** DBMS looks for the block in buffer pool.
- ➌ **Buffer Hit:** If found, read directly from memory.
- ➍ **Buffer Miss:** If not found, read block from disk into buffer.
- ➎ **Modify (Optional):** If data is updated, block becomes *dirty*.
- ➏ **Write Back:** Dirty blocks are later written back to disk.



DBMS uses a **Buffer Manager** to decide:

- 1 Which blocks to keep in buffer.
- 2 Which blocks to remove when space is full.

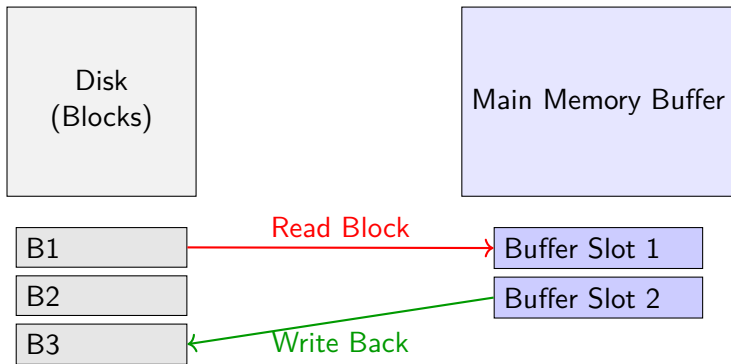
Example

Transaction: Update salary of Employee 105

- 1 Block containing record 105 is loaded into buffer.
- 2 Salary field is updated in buffer block.
- 3 Block marked as *dirty*.
- 4 Later, written back to disk during commit or buffer replacement.

Key Idea

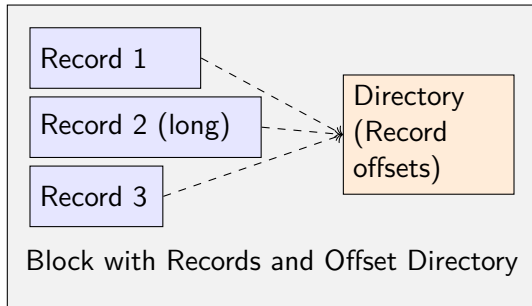
Most queries read and modify data in **buffers**, not directly on the disk — improving performance and reducing I/O cost.



- DBMS loads block B1 from disk to buffer for processing.
- When modified, the buffer is marked *dirty* and later written back.

- A file is divided into **blocks** (pages), and each block contains multiple **records**.
- Record placement affects:
 - Storage utilization.
 - Access speed.
- Types of record placement:
 - 1 **Fixed-Length Records:**
 - All records have equal size.
 - Easier to locate ($\text{offset} = \text{record size} \times \text{record number}$).
 - 2 **Variable-Length Records:**
 - Records differ in size (due to VARCHAR fields).
 - Need directory structure within block.

Block Structure for Variable-Length Records



- Directory at the end of block stores offsets to each record.
- Simplifies insertion and deletion.

- **Buffering** minimizes expensive disk I/O by keeping frequently used blocks in memory.
- **Block organization** determines how data is physically stored on disk.
- **Record placement** strategies (fixed or variable length) affect:
 - Space utilization
 - Access efficiency
 - Ease of update

Key Takeaway

Efficient buffering and record placement are essential for *fast query processing* and *optimal disk utilization*.

- 1 Introduction to Storage in DBMS
- 2 Secondary Storage Devices
- 3 Buffering of Blocks and Placing File Records on Disk
- 4 Operations on Files**
- 5 File Organisation - Heap, Sorted and Hash
- 6 RAID Technology
- 7 Indexing in DBMS
 - Example: Building a 3-Order B-Tree
- 8 Types of Indexes

- A **file** is a collection of related records stored on disk.
- The DBMS must perform various operations on these files to manage and access data efficiently.
- Common file operations:
 - 1 **Insert** – Add a new record.
 - 2 **Delete** – Remove a record.
 - 3 **Update** – Modify an existing record.
 - 4 **Search** – Locate a record based on some condition.
 - 5 **Scan** – Access all records sequentially.

Example

In an *EMPLOYEE* file, each record stores EmpID, Name, Dept, Salary. Operations like “find employee by ID” or “update salary” are performed on this file.

- To insert a record:
 - 1 Find a block with free space.
 - 2 Place the new record in that block.
 - 3 Update directory or header information.
- If no space is available:
 - Allocate a new block and link it to the file.

Insert new record here



Note

Insertion cost depends on the file organization (heap, sorted, hashed, etc.).

- To delete a record:
 - 1 Locate the record.
 - 2 Mark the record as deleted (do not always physically remove it).
 - 3 Optionally reuse the space for future insertions.
- Deleted records are sometimes kept as **tombstones** to maintain logical consistency.



Note

Physical deletion may cause fragmentation and requires compaction periodically.

- **Update:**

- Modify specific fields in an existing record.
- If new data increases record size, record may need to be moved to a new block.

- **Search:**

- Locate a record that matches a specific key or condition.
- Efficiency depends on file organization:
 - Heap file: Linear search.
 - Sorted file: Binary search.
 - Hashed file: Direct access.

Example

`SELECT * FROM Employee WHERE EmpID = 105;` → DBMS performs a search operation on the underlying file.

- **Scan** means accessing all records in the file sequentially.
- Required for:
 - Aggregate functions (e.g., AVG(Salary)).
 - Full-table queries.
 - Rebuilding indexes or reorganizing files.
- During scan:
 - 1 Each block is read into the buffer.
 - 2 All records inside the block are processed.
 - 3 Next block is fetched until EOF (End of File).

Performance Tip

Scanning can be parallelized using multiple buffers or disks (e.g., RAID).

- Basic file operations: Insert, Delete, Update, Search, Scan.
- Their performance depends on:
 - Type of file organization.
 - Buffer management efficiency.
 - Use of indexing and access paths.

Key Takeaway

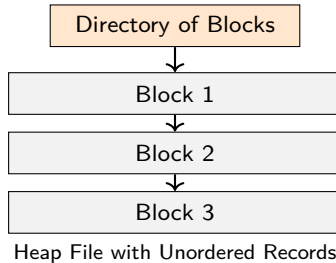
Efficient file operations are the foundation for query performance and transaction management in a DBMS.

- 1 Introduction to Storage in DBMS
- 2 Secondary Storage Devices
- 3 Buffering of Blocks and Placing File Records on Disk
- 4 Operations on Files
- 5 File Organisation - Heap, Sorted and Hash**
- 6 RAID Technology
- 7 Indexing in DBMS
 - Example: Building a 3-Order B-Tree
- 8 Types of Indexes

- A **Heap File** (also called *unordered file*) is the simplest type of file organization.
- Records are stored in the order they are inserted — no sorting or ordering based on any key.
- Each record has a unique **Record ID (RID)** used for access.

Definition

A **Heap File** stores records in arbitrary order, and new records are placed in the first available space.



- A **directory** maintains pointers to disk blocks containing data.
- Blocks are linked sequentially, and new records go into any block with space.

Insertion:

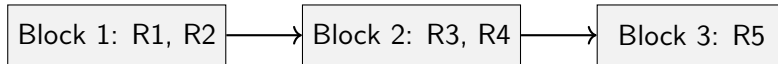
- Find the first block with available space.
- Place the new record there.
- Update the directory if the block becomes full.

Deletion:

- Locate the record via RID (Record ID).
- Mark it as deleted (lazy deletion) or remove it physically.

Performance Characteristics

Operation	Performance	Comments
Insert	$O(1)$	Very efficient
Delete	$O(1)$	Need RID
Search (by key)	$O(n)$	Linear search required
Scan	$O(n)$	Sequential block scan



Records inserted as they arrive

Observation

No particular order of records is maintained — hence, searching by attribute requires scanning the entire file.

Advantages:

- Simple to implement.
- Fast insertion and deletion.
- Efficient when most queries are *scans* or *insert-heavy*.

Disadvantages:

- Very slow search ($O(n)$).
- Records are not ordered.
- Poor performance for range queries.

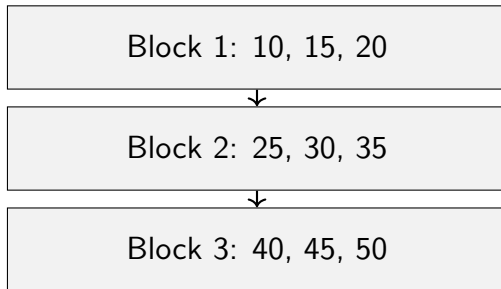
Use Case

Heap files are ideal for *temporary data*, *log files*, or *staging tables* where frequent insertions occur.

- In a **Sorted File** (also called *Sequential File*), records are stored in a specific order based on a **key field**.
- The sorting key is typically a unique attribute like EmpID, RollNo, or AccountNo.
- The order enables **fast searching** and efficient **range queries**.

Definition

A **Sorted File** stores records sequentially in ascending or descending order of a *search key*.



Records arranged by ascending key value

- Records are kept in sorted order of key.
- Access path: Sequential or binary search across blocks.

1. Insertion

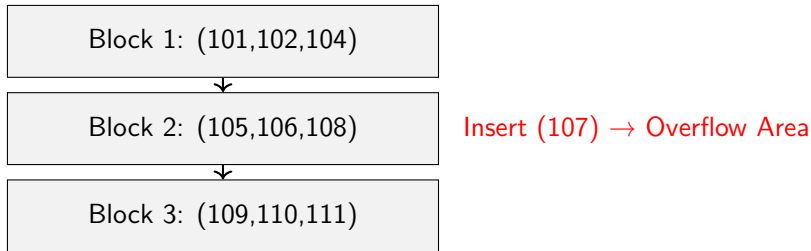
- Locate correct block using binary search.
- Insert record in order — may cause block overflow.
- Overflow handled using a *linked overflow area*.

2. Deletion

- Record is located and marked deleted.
- Periodic reorganization restores order and removes empty space.

3. Search

- Fast access using binary search on key field.
- *Range queries* can be answered efficiently by scanning consecutive blocks.



- Inserting 107 creates overflow in Block 2.
- Overflow record is linked externally until reorganization.

Performance Comparison

Operation	Heap File	Sorted File
Insert	Very Fast ($O(1)$)	Slower ($O(n)$)
Delete	Moderate	Moderate
Search (by key)	$O(n)$	$O(\log n)$
Range Query	$O(n)$	Efficient ($O(\log n + k)$)

- Best suited when:
 - Most queries are *search or range queries*.
 - Insertions and deletions are infrequent.

Advantages:

- Fast searching using binary search.
- Efficient for range-based queries.
- Ideal for reports and batch processing.

Disadvantages:

- Slow insertion and deletion (need to maintain order).
- Requires periodic reorganization.
- Not suitable for dynamic/real-time updates.

Example Use Case

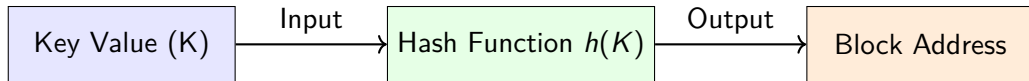
Sorted files are used in **bank ledgers**, **student result lists**, or **payroll systems** where data changes infrequently.

- **Hashing** provides **direct access** to records using a **hash function**.
- Instead of searching sequentially, the hash function computes the location (block number) of a record directly.
- Hashing is widely used for:
 - Fast **search**, **insert**, and **delete** operations.
 - Indexing and retrieval in large databases.

Concept

$$h(\text{Key}) = \text{Block Address}$$

The hash function maps a record's key to the address of the disk block where it is stored.



- The record is stored in the block given by $h(K)$.
- Example:

$$h(K) = K(\text{mod}10)$$

For Key = 12345 $\rightarrow h(12345) = 5 \rightarrow$ store in Block 5.

- In **Static Hashing**, the number of buckets (blocks) is fixed.
- The hash function maps keys into one of these predefined buckets.
- Each bucket stores multiple records.

Bucket 0: (40, 50)

Bucket 1: (11, 21)

Bucket 2: (12, 22)

Bucket 3: (33, 43)

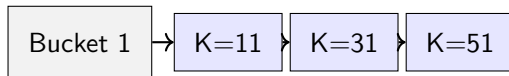
Example

For $h(K) = K(\text{mod}4)$, $K = 11 \rightarrow 3$, $K = 22 \rightarrow 2$, $K = 43 \rightarrow 3$.

- **Collision:** Two different keys hash to the same bucket.
- Must be resolved using a **collision resolution strategy**.

Common Techniques:

- 1 **Open Addressing:** Find another empty slot in the same file.
- 2 **Chaining:** Maintain a linked list of records that hash to the same address.



Chaining Example

All keys (11, 31, 51) hash to same bucket → maintained via a linked chain.

- In **Dynamic Hashing**, the number of buckets can grow or shrink as the database changes.
- It prevents overflow chains from growing too long.

Types:

- 1 **Extendible Hashing** – Uses a directory that grows dynamically.
- 2 **Linear Hashing** – Buckets split gradually as the file grows.

Extendible Hashing Concept

$h(K)$ = binary hash value

Directory (prefix)	Bucket
0	B0
1	B1
10	B2
11	B3

The directory expands as more bits of the hash value are used.

Time Complexity (Average Case)

Operation	Static Hashing	Dynamic Hashing
Search	$O(1)$	$O(1)$
Insert	$O(1)$	$O(1)$ (amortized)
Delete	$O(1)$	$O(1)$
Range Query	Poor	Poor

- Hashing is excellent for equality search (e.g., “find EmpID = 101”).
- Not suitable for range queries (e.g., “find salary between 10k–20k”).

Use Case

Hashing is ideal for **primary key lookups**, **index tables**, and **dictionary-based structures**.

Advantages:

- Very fast access ($O(1)$ expected time).
- Efficient insert and delete.
- No need to maintain order.

Disadvantages:

- Poor for range queries.
- Collisions cause overhead.
- Rehashing can be costly.

Example Use Case

Hashing is used in **database indexes**, **symbol tables**, and **hash joins**.

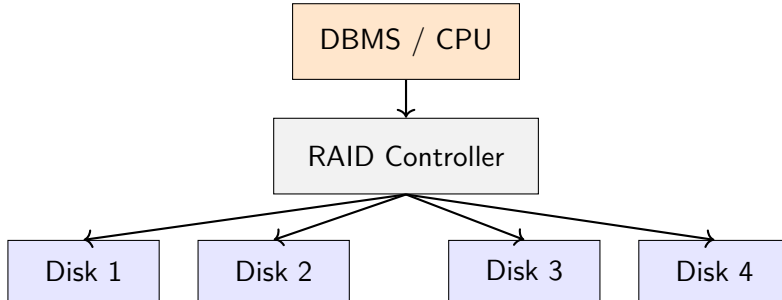
Feature	Heap File	Sorted File	Hashed File
Organization	Records stored in arbitrary order .	Records stored in sorted order based on key.	Records stored according to hash function output .
Insertion	Very fast; new record placed in first available space.	Slow; records must remain sorted.	Fast; uses hash function to find target bucket.
Deletion	Fast (mark record deleted).	Moderate (may need reorganization).	Fast; locate via hash, then remove.
Search (Equality)	Slow, linear search ($O(n)$).	Faster using binary search ($O(\log n)$).	Very fast, direct access ($O(1)$ expected).
Range Queries	Inefficient (scan entire file).	Highly efficient (sequential access).	Poor; hash does not preserve order.
Reorganization Need	Rarely needed.	Frequent (to maintain order).	When buckets overflow or during rehashing.
Space Utilization	Good, simple layout.	Moderate; may leave gaps after deletions.	May waste space due to overflow chains.
Use Cases	Temporary tables, logs, bulk loading.	Static datasets, reporting, range-based access.	Index tables, primary-key lookups, real-time systems.

- 1 Introduction to Storage in DBMS
- 2 Secondary Storage Devices
- 3 Buffering of Blocks and Placing File Records on Disk
- 4 Operations on Files
- 5 File Organisation - Heap, Sorted and Hash
- 6 RAID Technology**
- 7 Indexing in DBMS
 - Example: Building a 3-Order B-Tree
- 8 Types of Indexes

- **RAID** stands for **Redundant Array of Independent Disks**.
- It is a technique to use multiple disks together for:
 - ① **Improved performance** – by parallel data access.
 - ② **Higher reliability** – by storing redundant information.
- The main idea:
 - Split data across several disks (striping).
 - Add redundancy using parity or mirroring.

Definition

RAID combines multiple physical disks into a single logical unit to achieve a balance between performance, reliability, and storage cost.



Observation

The RAID controller manages how data is split and stored across disks, handling redundancy and recovery transparently.

Level	Technique	Advantages	Disadvantages
RAID 0	Data Striping (No Redundancy)	Very fast read/write, parallel access.	No fault tolerance; one disk failure causes data loss.
RAID 1	Mirroring (Duplication)	High reliability; data copied on two disks.	Expensive – storage cost doubles.
RAID 2	Bit-level Striping with Hamming Code	Error correction supported.	Rarely used; complex.
RAID 3	Byte-level Striping + Dedicated Parity Disk	Fast for large sequential access.	Parity disk bottleneck.
RAID 4	Block-level Striping + Dedicated Parity Disk	Random read fast.	Parity update slows down writes.
RAID 5	Block-level Striping + Distributed Parity	High throughput; fault-tolerant; efficient storage.	Slightly slower writes (parity calculation).
RAID 6	Block-level Striping + Double Parity	Survives 2 disk failures; high reliability.	Slower writes; higher parity overhead.

Illustration: RAID 0 (Data Striping)

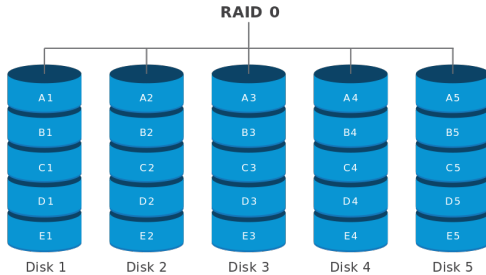


Figure: RAID 0

- Data is split across disks (A1 on Disk 1, A2 on Disk 2, etc.).
- Read and write can happen in parallel → High performance.

Limitation

No redundancy — if one disk fails, data is lost.

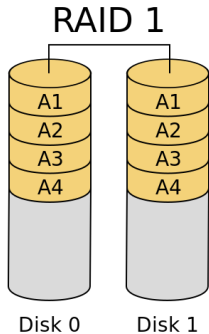


Figure: RAID 1

- Each disk has an exact copy of the other.
- Read performance doubles (either disk can serve requests).
- Write performance same as single disk.

Illustration: RAID 10 (Data Striping+Mirroring)

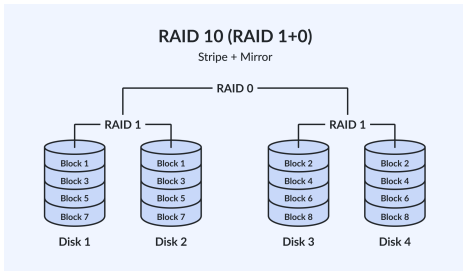


Figure: RAID 10

- RAID 10, also known as RAID 1+0, is a nested RAID configuration that combines RAID 1 (mirroring) and RAID 0 (striping) to provide both high performance and fault tolerance.
- It requires a minimum of four drives, and its main advantages are high I/O performance, making it ideal for databases and other I/O-intensive applications, and better performance than RAID 5/6 during rebuilds..
- The primary drawback is that it only provides 50% of the total raw drive capacity for usable storage.

Illustration: RAID 3 (Dedicated Parity)

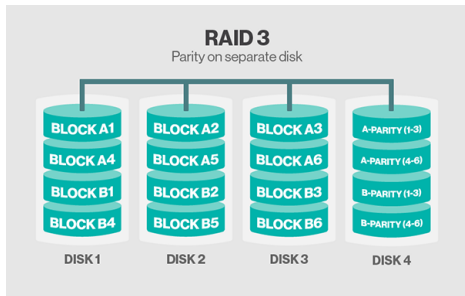
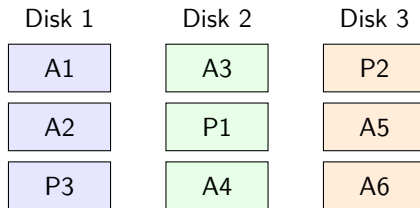


Figure: RAID 3

- RAID 3 is a data storage configuration that uses byte-level striping to distribute data across multiple drives, with a dedicated parity drive for data redundancy and fault tolerance.
- This setup offers high data transfer rates for large files by allowing parallel access to data across all drives, but can be limited in performance for simultaneous, small read operations due to the single parity disk.

Illustration: RAID 5 (Distributed Parity)



- Parity (P1, P2, P3) is distributed among disks.
- Balanced load and fault tolerance.

Illustration: RAID 6 (Dual xDistributed Parity)

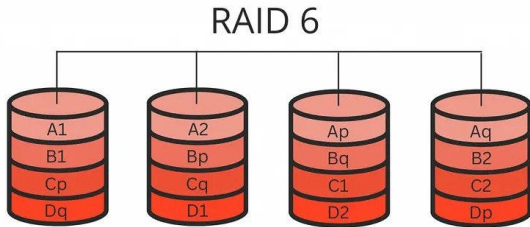
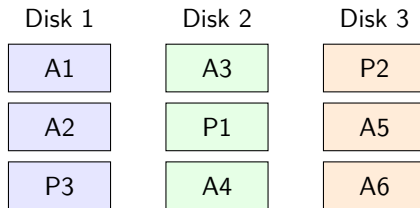


Figure: RAID 6

- RAID 6 is a storage configuration that uses data striping with dual distributed parity, allowing it to withstand the failure of any two drives in the array.
- It requires a minimum of four drives and provides high reliability at the cost of reduced write performance and a loss of storage capacity.
- This makes it suitable for environments where data protection is a higher priority than write speed.

Illustration: RAID 5 (Distributed Parity)



- Parity (P1, P2, P3) is distributed among disks.
- Balanced load and fault tolerance.

Performance Summary

RAID Level	Read Speed	Write Speed	Fault Tolerance
RAID 0	High	High	None
RAID 1	High	Moderate	Yes (1 disk)
RAID 5	High	Moderate	Yes (1 disk)
RAID 6	Moderate	Lower	Yes (2 disks)

- RAID increases performance through *parallel I/O*.
- Provides redundancy via parity or mirroring.

- RAID = Redundant Array of Independent Disks.
- Improves **speed** (through parallelism) and **reliability** (through redundancy).
- Common levels:
 - **RAID 0** – Striping (no redundancy)
 - **RAID 1** – Mirroring
 - **RAID 5** – Distributed parity (balance)

Key Takeaway

Modern DBMS use RAID for efficient **parallel disk access**, ensuring both high performance and data availability.

- 1 Introduction to Storage in DBMS
- 2 Secondary Storage Devices
- 3 Buffering of Blocks and Placing File Records on Disk
- 4 Operations on Files
- 5 File Organisation - Heap, Sorted and Hash
- 6 RAID Technology
- 7 Indexing in DBMS**
 - Example: Building a 3-Order B-Tree
- 8 Types of Indexes

Definition

- An **Index** is a data structure that improves the **speed of data retrieval** on a table.
- It creates a shortcut to access data quickly without scanning the whole table.
- Acts like a book index — you look up the topic and go directly to the page.

Examples

Without index: DBMS scans every row.

With index: DBMS jumps directly to the record location.

- To improve query performance and reduce I/O cost.
- Avoid full table scans for frequently accessed columns.
- Support faster searching, joining, and sorting operations.
- Automatically created on PRIMARY KEY or UNIQUE columns.

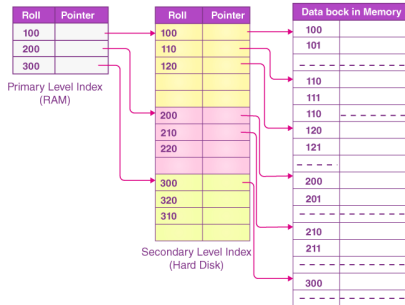
Example

Finding a student by RollNo in a 1 million record table:

- Without index → scan 1 million rows.
- With index → locate in few milliseconds ($\log n$ lookups).

Book Example:

- To find "Binary Trees" — you use the index page.
- Index lists the page number directly.
- Without it, you'd read the entire book!



(Indexing is like finding pages quickly using topic keywords.)

- Index maintains a **mapping** between key values and record addresses.

Index Table:

Key	Record Address
101	→ Address 1
104	→ Address 2
110	→ Address 3

Search Process:

- DBMS performs **binary search** on index keys.
- Finds the pointer → fetches actual record.
- Time complexity = $O(\log n)$.

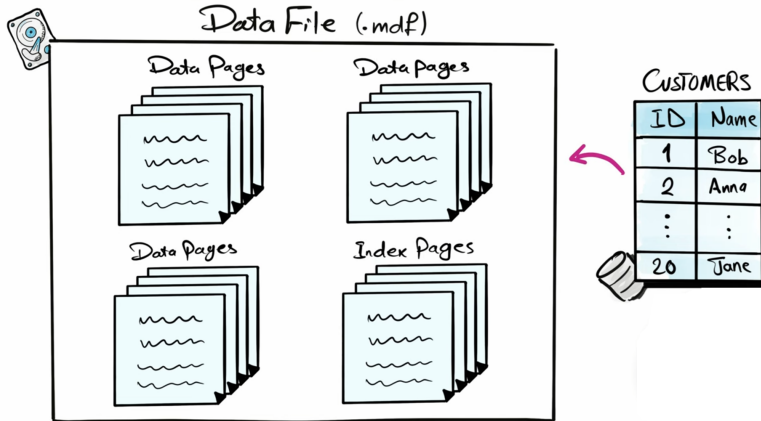


Figure: How the Data is Stored Physically in DBMS

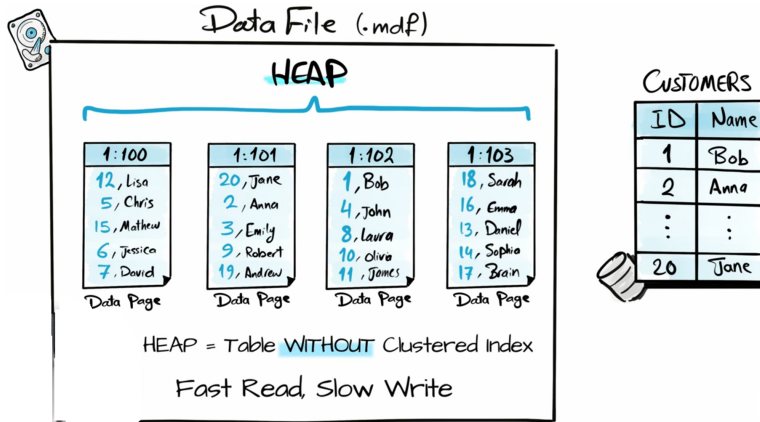


Figure: Data Storing Without Index

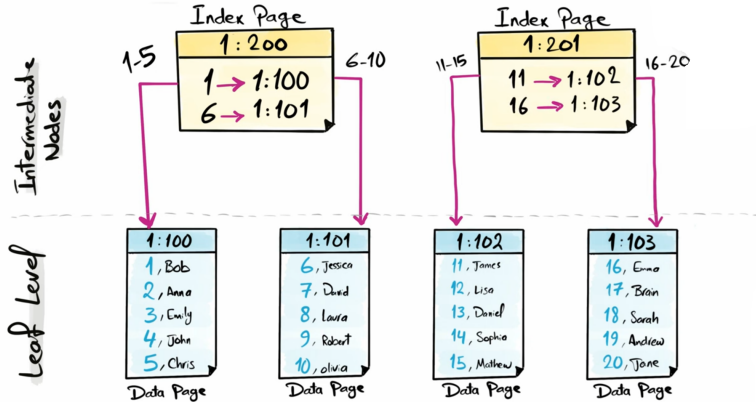


Figure: Data With Clustered Indexes

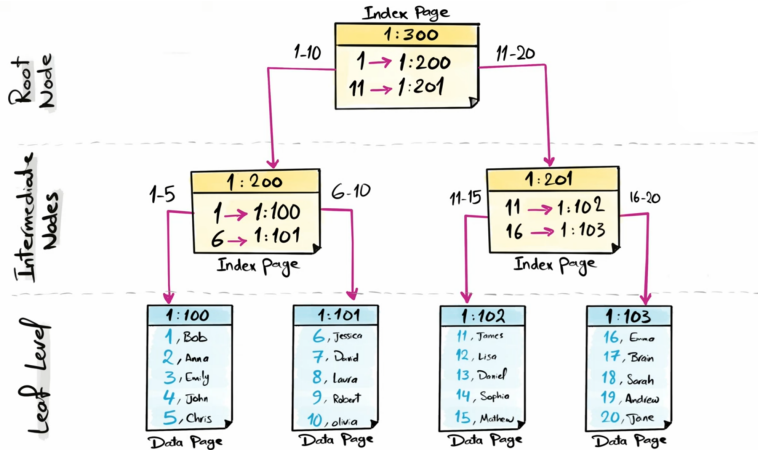


Figure: Data With Clustered Indexes

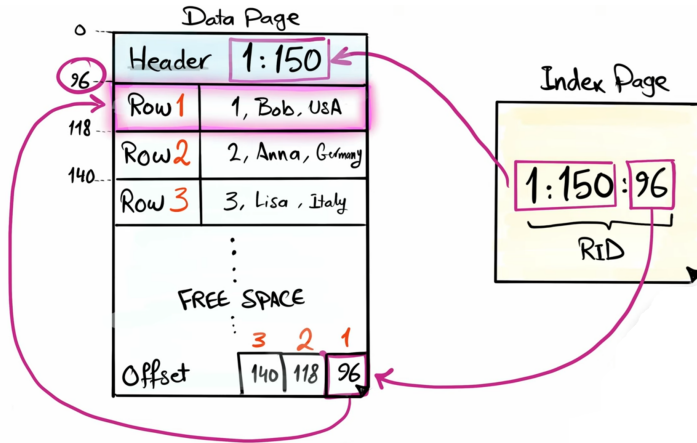


Figure: Data With Non-Clustered Indexes

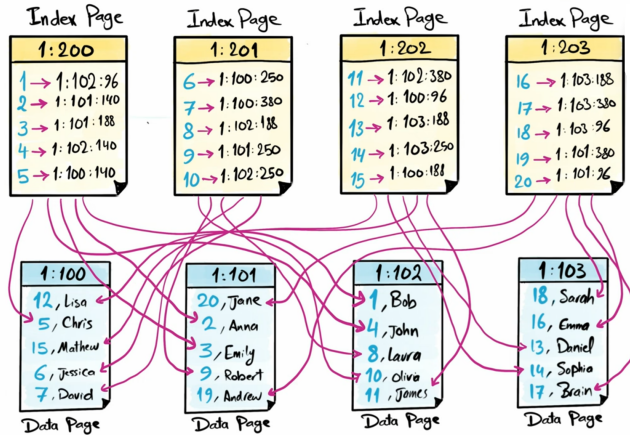


Figure: Data With Non-Clustered Indexes

B-Tree Index

- Balanced tree structure (used in MySQL, Oracle).
- Supports range queries: $<$, $>$, BETWEEN.
- Self-balancing $\rightarrow$ ensures $O(\log n)$ lookups.

Hash Index

- Based on hash table.
- Fast for $=$ (equality) lookups.
- Inefficient for range queries.

Examples

B-Tree is default in MySQL's InnoDB engine. Hash index is used in memory-optimized tables.

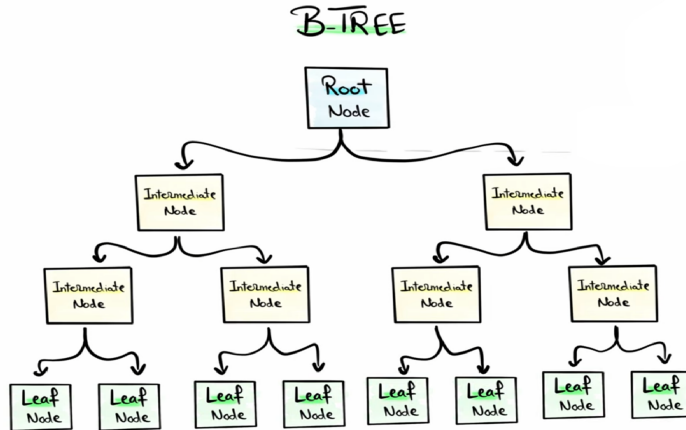


Figure: The btree

Definition

- Order 3 each node has at most 2 keys and 3 children.
- Internal nodes act as decision points between key ranges.
- The tree remains balanced after every insertion.

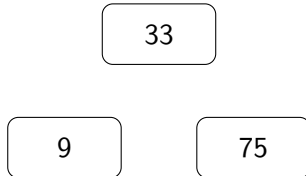
Examples

We will insert the sequence: 9, 33, 75, 41, 98, 214, 126, 55, 72

9, 33

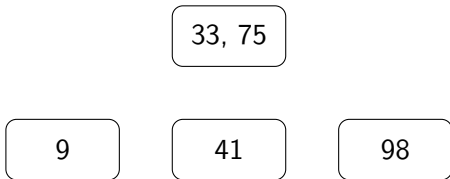
Status

Both keys fit in the root (max 2 keys). Tree height = 1.



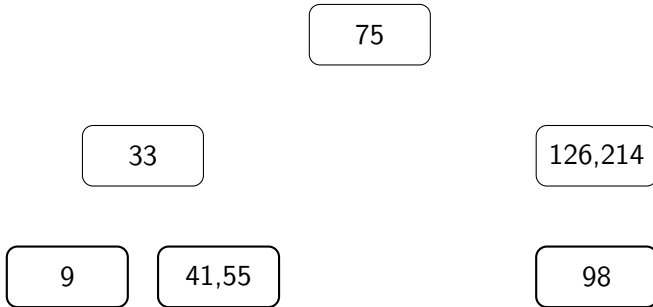
Explanation

Root $[9, 33]$ overflows after inserting 75 $\rightarrow$ split around 33. 33 becomes new root.



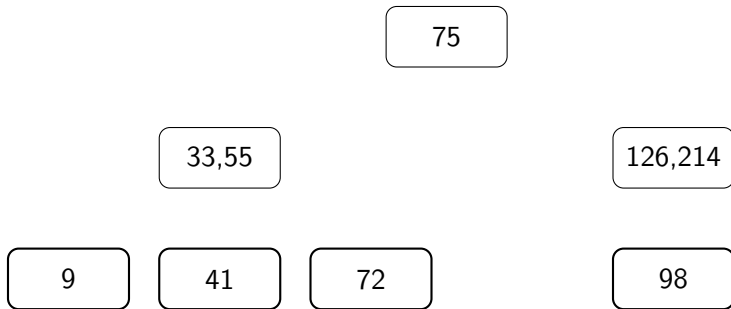
Explanation

Insert 41 → goes to middle child. Insert 98 → new right child created. Tree is balanced and valid.



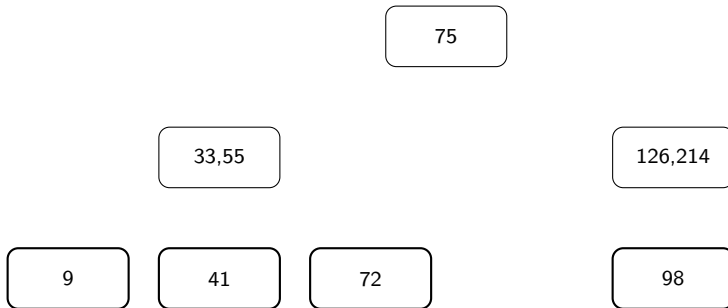
Explanation

Right child [98, 214] splits; median 126 moves up. 75 becomes new root.



Final Result

All leaves are at the same level. No node exceeds 2 keys. Tree height = 3.



Properties

- Root = 75
- Every node has 1–2 keys (3 children)
- Balanced, height = 3
- Basis of modern B⁺-Tree indexes in DBMS

- 1 Introduction to Storage in DBMS
- 2 Secondary Storage Devices
- 3 Buffering of Blocks and Placing File Records on Disk
- 4 Operations on Files
- 5 File Organisation - Heap, Sorted and Hash
- 6 RAID Technology
- 7 Indexing in DBMS
 - Example: Building a 3-Order B-Tree
- 8 Types of Indexes

- **Primary Index:** Automatically on PRIMARY KEY.
- **Unique Index:** Ensures values are unique.
- **Clustered Index:** Table data sorted physically by key.
- **Non-Clustered Index:** Separate structure pointing to table rows.
- **Composite Index:** Created on multiple columns.
- **Dense Index:** Has entry for every record.
- **Sparse Index:** Entries only for some records.

Feature	Clustered Index	Non-Clustered Index
Storage	Data stored physically in sorted order	Data and index stored separately
Count per Table	One	Many allowed
Speed	Faster for range queries	Slightly slower
Example	Primary key	Index on last_name

Examples

Clustered = physical order; Non-clustered = logical pointer to data.

```
-- Create an index
CREATE INDEX idx_emp_salary
ON employees (salary);

-- Create a unique index
CREATE UNIQUE INDEX idx_emp_email
ON employees (email);

-- Drop an index
DROP INDEX idx_emp_salary ON employees;
```

Examples

Indexes accelerate SELECT, JOIN, and ORDER BY queries.

Without Index:

```
SELECT * FROM employees WHERE emp_id = 987654;  
-- Full table scan ( $O(n)$ )
```

With Index:

```
CREATE INDEX idx_emp_id ON employees(emp_id);  
SELECT * FROM employees WHERE emp_id = 987654;  
-- Uses index lookup ( $O(\log n)$ )
```

Result: Dramatically faster retrieval.

```
CREATE INDEX idx_emp_name  
ON employees (last_name, first_name);
```

```
SELECT * FROM employees  
WHERE last_name='Sharma'  
AND first_name='Pooja';
```

Note

Composite index works best when the leftmost column (last_name) is part of the query condition.

```
SELECT e.first_name, d.dept_name  
FROM employees e  
JOIN departments d  
ON e.dept_id = d.dept_id;
```

Examples

To optimize this join:

```
CREATE INDEX idx_emp_deptid ON employees(dept_id);  
CREATE INDEX idx_dept_id ON departments(dept_id);
```

Result

Join execution becomes significantly faster because both tables can be searched via indexes.

Dense Index

- Entry for every record.
- More space, faster lookup.
- Used for small or frequently searched tables.

Sparse Index

- Entry for some records only.
- Smaller index file size.
- Efficient when data sorted on index key.

Hash Index

```
CREATE INDEX idx_emp_id_hash  
ON employees(emp_id) USING HASH;
```

- Excellent for equality lookups (=).
- Not suitable for ranges.

Bitmap Index

```
CREATE BITMAP INDEX idx_gender  
ON employees(gender);
```

- Best for low-cardinality columns.
- Common in data warehouses.

Advantages:

- Speeds up SELECT, JOIN, GROUP BY.
- Improves query optimization and sorting.

Disadvantages:

- Slows down INSERT, UPDATE, DELETE (extra writes).
- Consumes storage space.
- Must be rebuilt periodically to remove fragmentation.

- Index columns frequently used in:
 - WHERE clause
 - JOIN conditions
 - ORDER BY or GROUP BY
- Avoid indexing columns with few unique values (e.g., Gender).
- Rebuild or analyze indexes regularly for performance.

```
ALTER INDEX idx_emp_salary REBUILD;
```

Advantages

- Faster data retrieval.
- Improved join performance.
- Better query optimization.
- Enforces uniqueness.

Disadvantages

- Requires extra space.
- Overhead on updates.
- Maintenance cost.
- May degrade performance if misused.

- **Purpose:** Speed up data retrieval using key-value mapping.
- **Structures:** B-Tree, Hash, Bitmap.
- **Types:** Primary, Clustered, Non-Clustered, Composite.
- **Trade-off:** Faster reads, slower writes.
- **Best Use:** Columns in WHERE, JOIN, or ORDER BY.

Key Takeaway

Indexes = Performance + Precision Use them wisely for read-heavy workloads.

Thank You!

